

3F7 Information Theory and Coding

Full Technical Report

Liu Zihe
zl559

University of Cambridge
December 6 2025

1 Introduction

Information theory provides the mathematical framework for quantifying information and establishing fundamental limits on data compression. Shannon’s Source Coding Theorem establishes the entropy $H(X)$ as the lower bound for the average code length of an independent and identically distributed (i.i.d.) source. While i.i.d. models are useful first approximations, real-world data sources—such as natural language—exhibit strong temporal dependencies (memory) and non-uniform distributions.

This report extends the investigation of the 3F7 short laboratory in two directions. First, we analyse **discrete stationary sources**, relaxing the i.i.d. assumption to model sources with memory. We investigate the *entropy rate* H_∞ of a stochastic process and the convergence properties of block entropies. Second, we explore **adaptive coding**, where the probability model is learned online instead of 2 separate phases (building the probability model from the complete text and encoding). We derive the Laplace estimator from Bayesian principles and evaluate the performance of adaptive arithmetic coding against static methods.

2 Overview of Short Lab

The short laboratory focused on compressing `hamlet.txt` (approx. 207kB using fixed-length ASCII encoding) using three classical techniques designed for i.i.d. sources.

Shannon-Fano Coding constructs prefix-free codes by assigning lengths $l_i = \lceil \log_2(1/p_i) \rceil$. While it guarantees unique decodability (via the Kraft inequality $\sum 2^{-l_i} \leq 1$), it is suboptimal because it ignores the combinatorial nature of packing the prefix-free code tree (codewords can still be placed at shorter leaf nodes, hence optimizing our encoding)

Huffman Coding is the optimal symbol-by-symbol prefix code. It builds a binary tree bottom-up by iteratively merging the two least probable symbols. Our experiments confirmed that its expected length L satisfies $H(X) \leq L < H(X) + 1$. However, its performance is constrained by the requirement that each symbol must use an integer number of bits.

Arithmetic Coding overcomes the integer constraint by encoding entire sequences as sub-intervals of $[0, 1)$. By mapping a sequence x^n to an interval of width $P(x^n)$, it achieves a code length $L \approx -\log_2 P(x^n)$ bits. For large n , the per-symbol length converges to the entropy $H(X)$. This method proved superior for longer blocks but required accurate probability models.

The limitation of all three methods in the short lab was the assumption that text is a memoryless source, ignoring the significant redundancy in English grammar and spelling.

3 Coding for Discrete Stationary Sources

3.1 Theoretical Background

We model natural language text as a sequence of random variables, specifically a *discrete stochastic process*, $\mathcal{X} = \{X_i\}_{i=1}^{\infty}$. We distinguish between two primary classes of sources:

Memoryless Sources (i.i.d.): The simplest model assumes that successive symbols are i.i.d. A source is *memoryless* if the joint probability of a sequence factorizes completely into the product of its components:

$$\mathbb{P}(X_1, \dots, X_n) = \prod_{i=1}^n \mathbb{P}(X_i). \quad (1)$$

For such a source, the information content is additive, and the entropy of a block is simply $H(X^n) = nH(X)$. The optimal code length per symbol is thus fixed at the marginal entropy $H(X)$. The fixed-length ASCII and standard Huffman coding (without blocking) implicitly assume this model.

Stationary Sources (Sources with Memory): Natural languages like English are not memoryless; the letter 'h' is much more likely to follow 't' than 'q'. To capture this, we relax the independence assumption while retaining time-invariance. A process is *strictly stationary* if the joint distribution of any subsequence is invariant to time shifts. That is, for any shift τ and block length n :

$$\mathbb{P}(X_1 = x_1, \dots, X_n = x_n) = \mathbb{P}(X_{1+\tau} = x_1, \dots, X_{n+\tau} = x_n). \quad (2)$$

Stationarity implies that marginal probabilities are constant $H(X_i) = H(X_1)$ and conditional probabilities depend only on the relative history, not the absolute time.

Entropy Rate: For a source with memory, the marginal entropy $H(X_1)$ overestimates the true information content because it ignores the predictability provided by the context. The fundamental limit is instead given by the *entropy rate* H_{∞} , defined as the asymptotic per-symbol entropy [1, Def 4.1]:

$$H_{\infty} = \lim_{n \rightarrow \infty} \frac{1}{n} H(X_1, \dots, X_n). \quad (3)$$

This rate represents the *irreducible randomness* of the process after all past history is accounted for. To estimate H_{∞} , we define two sequences of block entropies:

$$H_n = \frac{1}{n} H(X_1, \dots, X_n) \quad (\text{Entropy per symbol of } n\text{-block}) \quad (4)$$

$$H_n^* = H(X_n | X_{n-1}, \dots, X_1) \quad (\text{Conditional entropy}) \quad (5)$$

3.2 Convergence of Entropy Rates (Theorem 1)

For stationary sources, these sequences converge to the entropy rate H_{∞} as $n \rightarrow \infty$. We provide proofs for the properties stated in the FTR handout, referencing Cover & Thomas [1, Chap 4].

Statement 1: $H_{n+1}^* \leq H_n^*$ (Conditional entropy is non-increasing).

Proof:

$$\begin{aligned} H_{n+1}^* &= H(X_{n+1} | X_n, \dots, X_2, X_1) \\ &\leq H(X_{n+1} | X_n, \dots, X_2) \quad (\text{Conditioning reduces entropy}) \\ &= H(X_n | X_{n-1}, \dots, X_1) \quad (\text{By Stationarity}) \\ &= H_n^* \quad \square \end{aligned}$$

Statement 2: $H_{n+1} \leq H_n$ (Per-symbol entropy is non-increasing).

Proof: By the chain rule, $H(X_1, \dots, X_{n+1}) = H(X_1, \dots, X_n) + H(X_{n+1}|X_1, \dots, X_n)$. Then

$$H_{n+1} = \frac{H(X_1, \dots, X_{n+1})}{n+1} = \frac{nH_n + H_{n+1}^*}{n+1} \quad (6)$$

Since $H_{n+1}^* \leq H_n^*$ (from Statement 1), $H_n^* \leq H_n$ (from statement 3), then $H_{n+1}^* \leq H_n^* \leq H_n$ where H_n is the average of joint entropy across the n terms. Adding a term H_{n+1}^* that is smaller than or equal to the previous average must reduce (or maintain) the new average. Thus $H_{n+1} \leq H_n$. $\square$

Statement 3: $H_n^* \leq H_n$.

Proof: From the Chain Rule for entropy,

$$H(X_1, \dots, X_n) = \sum_{i=1}^n H(X_i|X_{i-1}, \dots, X_1). \quad (7)$$

Dividing by n :

$$H_n = \frac{1}{n} \sum_{i=1}^n H_i^*. \quad (8)$$

Since H_i^* is a non-increasing sequence (Statement 1), the current term H_n^* is less than or equal to all preceding terms H_i^* for $i < n$. Therefore, the average H_n must be greater than or equal to the minimum term H_n^* . $\square$

Statement 4: $\lim_{n \rightarrow \infty} H_n = \lim_{n \rightarrow \infty} H_n^* = H_\infty$.

Proof: Since H_n^* is non-increasing (Statement 1) and bounded below by 0 (entropy is non-negative), the limit $\lim_{n \rightarrow \infty} H_n^* = H'$ exists. Since H_n is the arithmetic mean (Cesàro mean) of the sequence H_n^* as $n \rightarrow \infty$, by the *Cesàro Mean Theorem* (if $a_n \rightarrow a$, then $\frac{1}{n} \sum a_i \rightarrow a$), the sequence of averages converges to the same limit [1, Thm 4.2.1]. This limit is defined as the *entropy rate* H_∞ . $\square$

3.3 Experimental Results: Entropy Rate Convergence

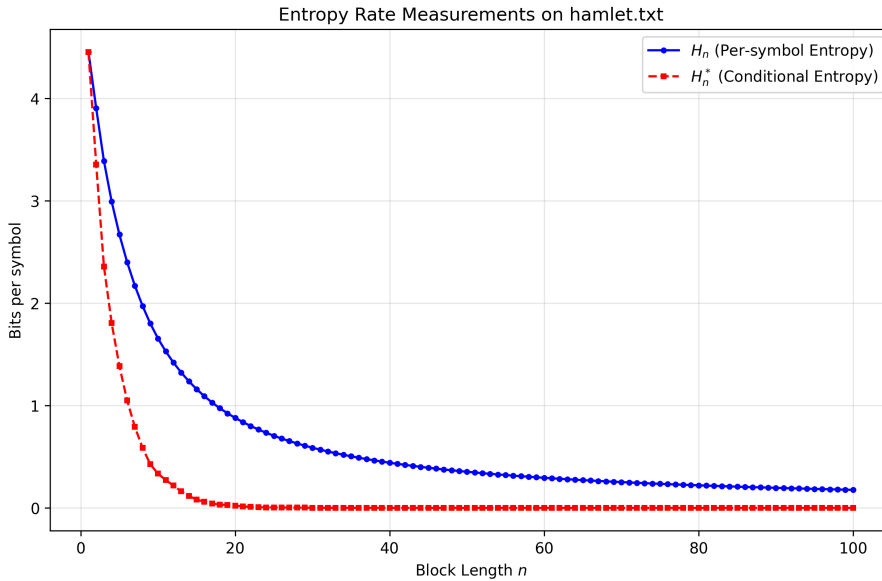


Figure 1: Entropy rate measurements on `hamlet.txt`. The estimates converge to 0 for large n due to data sparsity, not because the text is deterministic.

We measured the block entropy per symbol, $H_n = \frac{1}{n}H(X_1, \dots, X_n)$, for $n = 1$ to 100. As shown in Fig. 1, estimates for small n ($n = 1 - 20$) decrease quickly, correctly reflecting that English text is a source with memory; knowledge of previous characters reduces the uncertainty of the current one. However, for $n \geq 4$, the estimate continues to decay toward zero. The true entropy rate of English (H_∞) is surely not zero. Rather, this decay is a statistical artifact of a maximum likelihood estimator (of the entropy rate) operating in a high-dimensional regime with insufficient data (the "curse of dimensionality"). Accurate measurement of H_∞ is impossible without a sample size N that scales as $|\mathcal{X}|^n$.

3.4 Implications for Static Coding

The convergence failure observed in highlights a critical limitation for static source coding. To approach the entropy rate of a stationary source, the encoder must exploit a context of length n . In a static framework, this requires transmitting the probability model (e.g., a frequency table) to the decoder as side information. The size of this model is proportional to $|\mathcal{X}|^n$. As we increase n to capture longer dependencies, this overhead grows exponentially, quickly exceeding the size of the compressed data itself. Consequently, static methods cannot simply let $n \rightarrow \infty$; they are fundamentally limited to a small, finite n .

3.5 Context-Adaptive Arithmetic Coding

To overcome the static overhead limit, we employ *Context-Adaptive Arithmetic Coding*, often formalized as *Prediction by Partial Matching* (PPM). Instead of transmitting a massive static table, the model consists of a set of conditional probability distributions $P(X_t|C)$ that are built adaptively during encoding, where C is the context ($X_{t-n}, \dots, X_{t-1}$).

This approach directly addresses the data sparsity problem. When a symbol appears in a context C that has zero frequency (has never been seen before), encoder and decoder then "back off" to a shorter context of length $n - 1$, recursing down to order-0 if necessary. By using this escape mechanism, the algorithm can exploit very large n for frequent patterns (where statistics are rich) while falling back to shorter contexts for rare sequences. This allows the compression rate to approach H_∞ without the prohibitive $|\mathcal{X}|^n$ overhead required by static methods.

4 Adaptive Coding

4.1 Bayesian Derivation of the Laplace Estimator

In adaptive coding, we face the "zero-frequency problem": if a symbol x has not yet appeared in the stream, its Maximum Likelihood Estimate (MLE) is $P(x) = 0$. This is catastrophic for arithmetic coding. To solve this, we must assign probability mass to unseen events. We derive the solution using Bayesian inference, which treats the unknown probabilities not as fixed values, but as random variables describing our uncertainty [3, p. 97].

4.1.1 The Beta Posterior

Consider a binary version of the problem where we simply estimate the probability θ that the next symbol is a specific character 'x'. We start with **Bayes' Theorem**:

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)} \quad (9)$$

where D is the observed data (sequence of symbols), $P(\theta)$ is the *prior* belief, and $P(D|\theta)$ is the *likelihood* of the data.

- **Prior:** Since we assume no prior knowledge about the text, we must choose a prior that represents total ignorance. For a probability parameter $\theta \in [0, 1]$, the natural choice is the Uniform Distribution, also known as the $Beta(1, 1)$ distribution [2, Ch 3]. In the Bayesian framework, this corresponds to a prior which expresses no initial preference for one parameter set over another:

$$P(\theta) = 1 \quad \text{for } 0 \leq \theta \leq 1 \quad (10)$$

- **Likelihood:** Suppose we observe a stream of length n containing n_x occurrences of 'x' and n_y occurrences of other symbols ($n_y = n - n_x$). Assuming independence given θ , the probability of seeing this specific sequence is:

$$P(D|\theta) = \theta^{n_x} (1 - \theta)^{n_y}. \quad (11)$$

- **Posterior:** Multiplying the prior and likelihood gives us the posterior distribution (proportionality ignores the normalizing constant $P(D)$):

$$P(\theta|D) \propto \theta^{n_x} (1 - \theta)^{n_y} \cdot 1. \quad (12)$$

This resulting distribution is a Beta distribution with parameters $(n_x + 1, n_y + 1)$. (The general form of the Beta probability density function is: $f(\theta; \alpha, \beta) \propto \theta^{\alpha-1} (1 - \theta)^{\beta-1}$) As n grows, this curve becomes a sharp peak around the empirical frequency n_x/n , representing our growing confidence.

4.1.2 Laplace's Rule of Succession (Binary Case)

To encode the *next* symbol X_{n+1} , we do not simply use the most probable value of θ (the peak). Instead, we must average over *all possible* values of θ , weighted by their posterior probability - a process known as *marginalization*. This predictive probability is equivalent to the expected value of the posterior distribution:

$$P(X_{n+1} = x|D) = \int_0^1 P(X_{n+1} = x|\theta) P(\theta|D) d\theta \quad (13)$$

$$= \int_0^1 \theta \cdot P(\theta|D) d\theta \quad \text{since } P(X_{n+1} = x|\theta) = \theta \text{ by definition} \quad (14)$$

$$= \mathbb{E}[\theta|D] \quad (15)$$

This expectation corresponds to the Minimum Mean-Squared Error (MMSE) estimator in Bayesian theory. Since our posterior is a Beta distribution $Beta(\alpha, \beta)$ with $\alpha = n_x + 1$ and $\beta = n_y + 1$, we use the known formula for the mean of a Beta distribution, $\mu = \frac{\alpha}{\alpha + \beta}$ [2, 4]:

$$\hat{p}(x) = \mathbb{E}[\theta|D] = \frac{\alpha}{\alpha + \beta} = \frac{n_x + 1}{(n_x + 1) + (n_y + 1)} = \frac{n_x + 1}{n + 2}. \quad (16)$$

This result is Laplace's Rule of Succession. It effectively adds 1 "pseudocount" to both outcomes, preventing zero probabilities [2, p. 52].

4.1.3 Generalization to Text (The Dirichlet Distribution)

We now generalize from binary outcomes to an alphabet of size K (e.g., $K = 256$ for extended ASCII). The unknown parameter is now a probability vector $\theta = (\theta_1, \dots, \theta_K)$ where $\sum_{i=1}^K \theta_i = 1$.

The natural generalization of the Beta distribution to K dimensions is the *Dirichlet distribution* [2, pg 317]. Just as the Beta distribution represents our uncertainty about a single probability

parameter, the Dirichlet distribution represents our uncertainty about a *vector* of probabilities that must sum to 1. The Dirichlet distribution with parameters $\alpha = (\alpha_1, \dots, \alpha_K)$ has probability density [2, Eq. 23.30]:

$$P(\theta|\alpha) = \frac{1}{B(\alpha)} \prod_{i=1}^K \theta_i^{\alpha_i-1} \quad (17)$$

where $B(\alpha) = \frac{\prod_i \Gamma(\alpha_i)}{\Gamma(\sum_i \alpha_i)}$ is the multivariate Beta function (normalizing constant) and each $\alpha_i > 0$ [5].

Suppose we observe a sequence with symbol counts $\mathbf{n} = (n_1, \dots, n_K)$ where $N = \sum_i n_i$ is the total number of observations.

- **Prior:** We start with a uniform prior representing complete ignorance, which corresponds to $\alpha = (1, \dots, 1)$:

$$P(\theta) = \text{Dir}(1, \dots, 1) = \text{Uniform} \quad (18)$$

- **Likelihood:** The probability of observing these counts follows the multinomial distribution:

$$P(\mathbf{n}|\theta) \propto \prod_{i=1}^K \theta_i^{n_i} \quad (19)$$

- **Posterior:** By Bayes' theorem, the posterior is also Dirichlet with updated parameters:

$$P(\theta|\mathbf{n}) = \text{Dir}(\alpha_1 + n_1, \dots, \alpha_K + n_K) = \text{Dir}(1 + n_1, \dots, 1 + n_K) \quad (20)$$

To predict the next symbol, we must marginalize over all possible parameter values weighted by the posterior. This predictive probability equals the *mean* of the posterior Dirichlet distribution. For a Dirichlet distribution with parameters $(\alpha'_1, \dots, \alpha'_K)$, the mean is [2, Eq. 23.32]:

$$\mathbb{E}[\theta_i] = \frac{\alpha'_i}{\sum_{j=1}^K \alpha'_j} \quad (21)$$

Substituting our posterior parameters $\alpha'_i = 1 + n_i$:

$$\hat{p}(i) = \mathbb{E}[\theta_i|\mathbf{n}] = \frac{n_i + 1}{\sum_{j=1}^K (n_j + 1)} = \frac{n_i + 1}{N + K} \quad (22)$$

This is the *Laplace estimator*. It ensures every symbol has non-zero probability of at least $1/(N + K)$, solving the zero-frequency problem. The uniform prior $\alpha = (1, \dots, 1)$ effectively adds 1 "pseudocount" to each symbol, which can be interpreted as pretending we observed each symbol once before seeing any real data [5, 2].

4.2 Experimental Performance and Analysis

We simulated adaptive arithmetic coding on `hamlet.txt`. The code length L for a sequence $x_1, \dots, x_n$ is estimated as the sum of the log-probabilities assigned by the model at each step:

$$L = - \sum_{i=1}^n \log_2 \hat{p}_{x_i} \quad (23)$$

Note that the probability model here built is a still a 0-order adaptive model (we are still learning the distribution assuming each character is i.i.d.) The model is adaptive in the sense of updating its probabilities as more characters are observed.

4.2.1 Convergence to Static Entropy

Figure 2 shows the running average rate L/n , which is the estimate of the entropy of the i.i.d distribution.

- **Initial Behaviour:** For small n , the rate is high (> 6 bits) because counts are low and the uniform prior dominates the distribution.
- **Asymptotic Behavior:** As $n \rightarrow \infty$, the estimator $\hat{p}_x$ converges to the true relative frequencies of the file. Consequently, the adaptive rate converges to the static i.i.d. entropy H_1 (≈ 4.45 bits). Note that the curve stays slightly above H_1 . This redundancy represents the “learning cost” of building the model online (we haven’t observed the complete data yet)

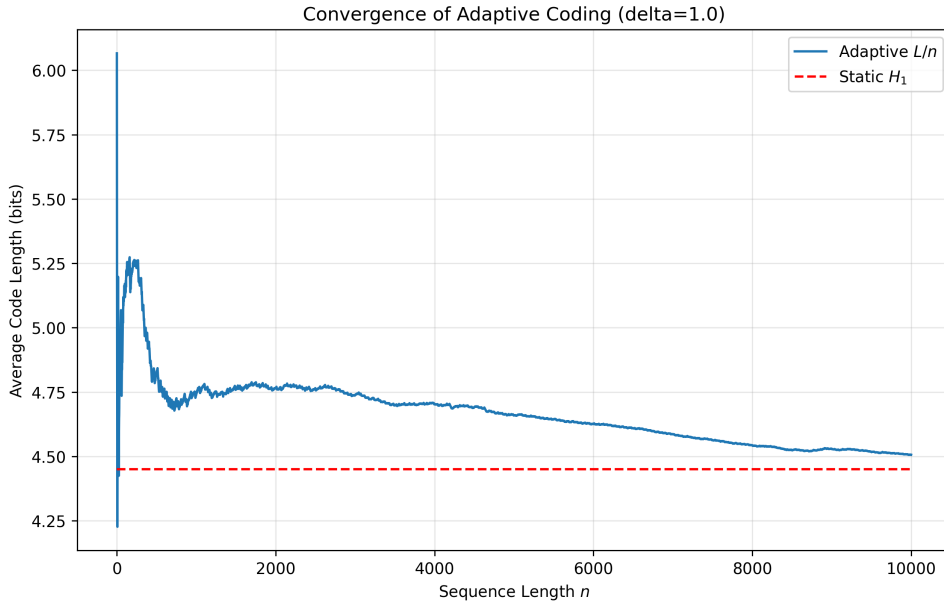


Figure 2: Convergence of 0-order Adaptive Coding (L/n) to the static entropy H_1 with $\delta = 1.0$ (Laplace Estimator) over $N = 10,000$ symbols.

4.2.2 Sensitivity to Bias Parameter δ

We investigated the impact of the pseudocount parameter δ by generalizing the estimator to:

$$\hat{p}(x) = \frac{n_x + \delta}{N + K\delta} \quad (24)$$

We tested $\delta \in [0.01, 50]$ on the first $N = 10,000$ symbols of `hamlet.txt` to visualize convergence behavior (Fig. 3), and then measured the final compression rate on the *entire text* ($N = 207,039$) to determine the optimal parameter (Fig. 4).

- **Small δ ($\delta < 0.5$):** When δ is very small (e.g., 0.01), the model assigns negligible probability mass to unseen characters. This makes the “zero-frequency” penalty extremely harsh. During the first appearance of a symbol ($n_x = 0$), the estimated probability is $P \approx \delta/N$, leading to a code length of $-\log_2(\delta/N)$. As $\delta \rightarrow 0$, this initial cost spikes dramatically, as seen in the sharp initial peaks in Fig. 3. However, once a symbol is seen, the model adapts very quickly.
- **Large δ ($\delta > 1.0$):** A large δ (e.g., 50.0) corresponds to a strong prior belief that the distribution is uniform. It dominates the denominator $N + K\delta$, diluting the influence of

Adaptive Coding Convergence vs. Prior Strength δ
($N=10000$ symbols)

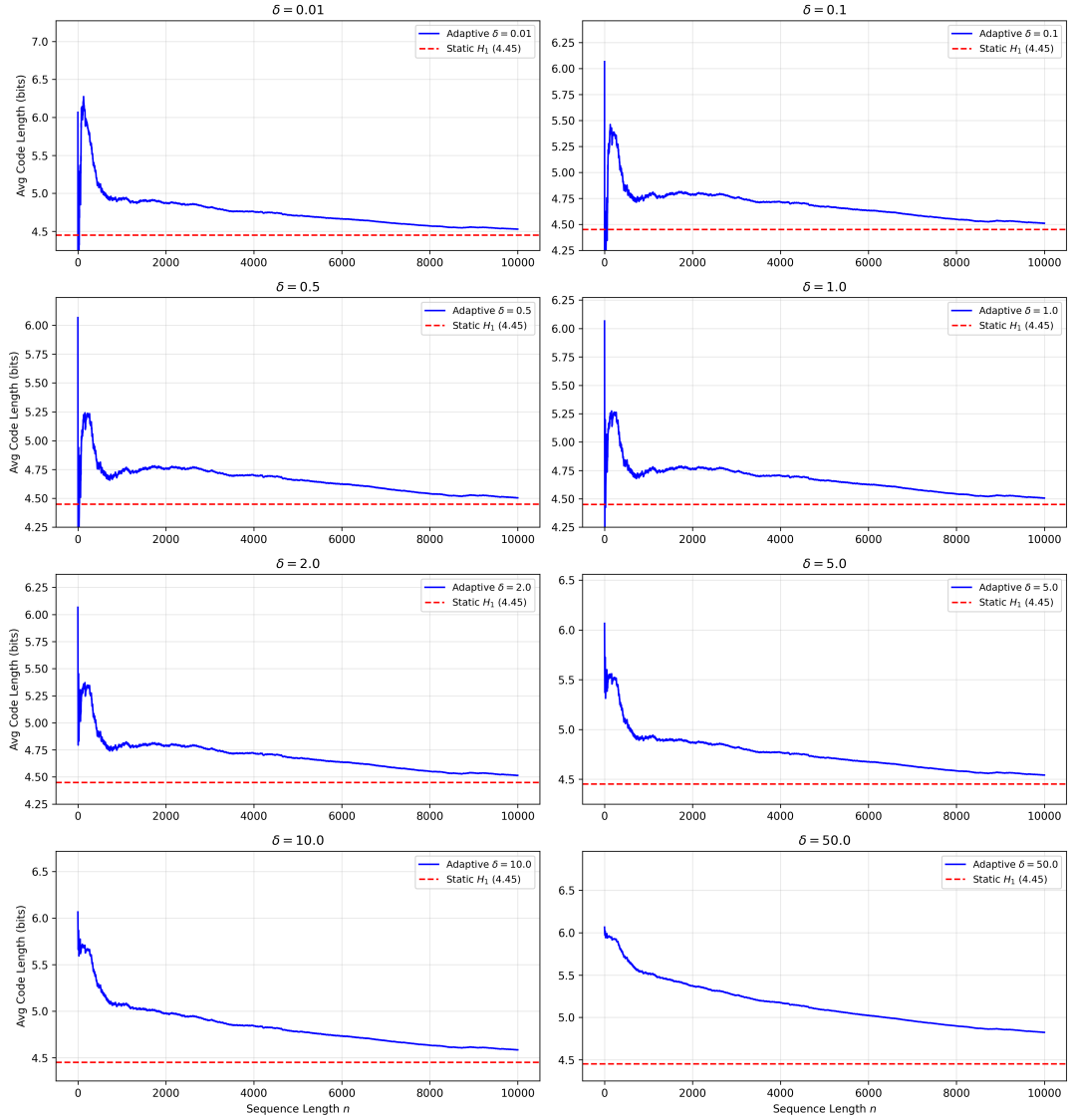


Figure 3: Adaptive coding convergence profiles for various prior strengths δ ($N = 10,000$ symbols). Small δ leads to rapid learning but high volatility; large δ creates a stable but slow-converging model.

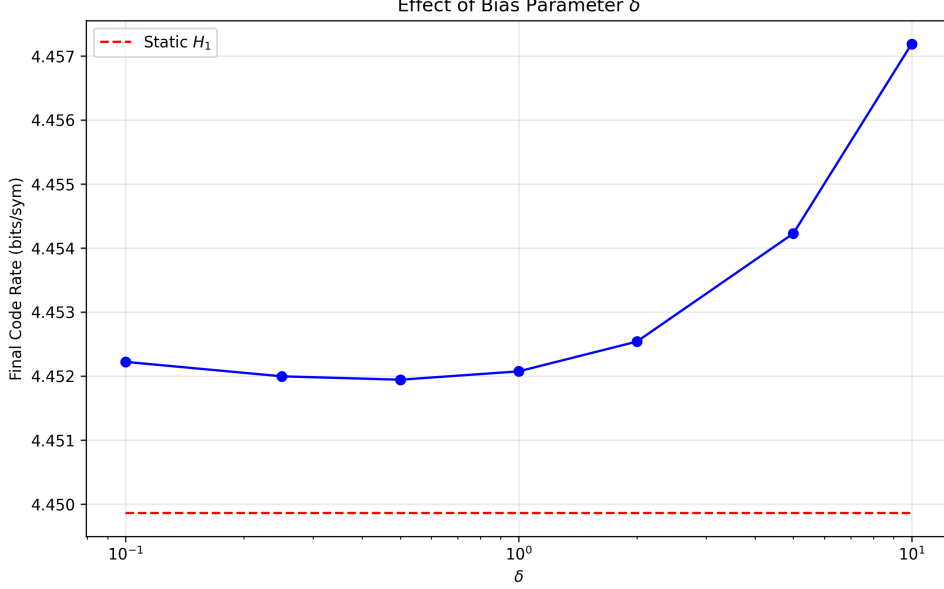


Figure 4: Effect of bias δ on the final compression rate, measured on the *entire* Hamlet text ($N \approx 207k$)

the observed data counts n_x . This acts as a “dampener,” resulting in a very stable curve that converges slowly to the true entropy (Fig. 3, bottom right).

- **Optimal $\delta = 0.5$:** Our experiments on the full text (Fig. 4) show that $\delta = 0.5$ yields the minimum code rate, slightly outperforming the Laplace estimator ($\delta = 1$)

4.3 Comparison of Adaptive vs. Static Coding

To rigorously compare the performance of Adaptive and Static coding, we invoke the *Minimum Description Length (MDL)* principle.¹ This principle states that the total description length $L(x_1 \dots x_n) = L(x^n)$ is composed of the cost to encode the data given a model, plus the cost of specifying the probability model itself:

$$L(x^n) = L(\text{Data} \mid \text{Model}) + L(\text{Model}) \quad (25)$$

In our experiment, we compare the cumulative code lengths of the two approaches as a function of the sequence length n :

Static Coding (Theoretical Bound): Since static coding requires a pre-computed frequency table, it is impossible to run “online” in the same manner as adaptive coding. Instead, we plot the theoretical lower bound at each length n . This bound assumes that for any prefix of length n , we transmit the optimal frequency table followed by the data. The cost is calculated using the standard MDL penalty [2, Ch. 28]:

$$L_{\text{static}}(n) \approx \underbrace{nH_n}_{\text{Data Cost}} + \underbrace{\frac{K-1}{2} \log_2 n}_{\text{Model Cost}} \quad (26)$$

The term $\frac{K-1}{2} \log_2 n$ represents the parametric complexity²

¹Interestingly enough, I actually wrote a paper using the Bayesian Information Criterion, so I’m delighted to see its use here. See <https://arxiv.org/abs/2409.04913>

²The “Occam Factor”, the cost of specifying $K - 1$ independent probability parameters to a precision of $1/\sqrt{n}$

Adaptive Coding (Actual Cost): This curve represents the number of bits generated by the simulated arithmetic coder up to symbol n . Unlike the static case, there is no explicit model header ($L(\text{Model}) = 0$). Instead, the cost is the sum of the negative log-probabilities assigned by the sequential estimator:

$$L_{\text{adaptive}}(n) = \sum_{i=1}^n -\log_2 \hat{P}(x_i | x^{i-1}) \quad (27)$$

The "redundancy" in this case is the difference between this sum and the ideal entropy nH_n . This difference represents the "learning penalty" incurred while the estimator $\hat{P}$ converges from the uniform prior to the true statistics.

4.3.1 Experimental Results

Figure 5 presents the comparison of these two quantities on the `hamlet.txt` sequence.

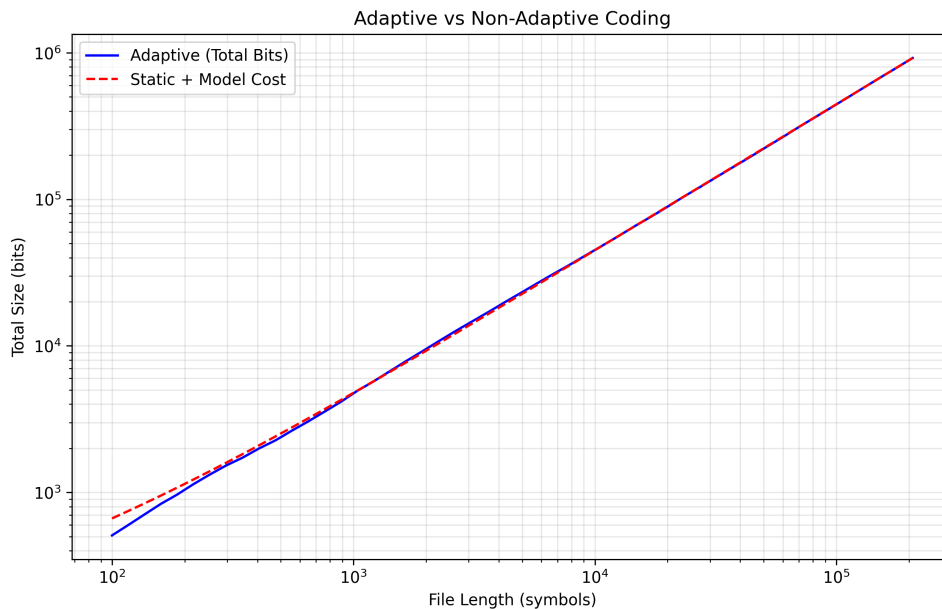


Figure 5: Comparison of Adaptive Coding (actual cumulative bits) versus Static Coding (theoretical data + MDL model cost). The curves run parallel for large n , confirming that the adaptive learning penalty scales identically to the static model overhead.

- **Short Sequences ($n < 10^4$):** Adaptive coding is superior. The static method suffers from the "overhead problem," where the fixed cost of the frequency table ($\approx \frac{K}{2} \log n$) outweighs the data payload. Adaptive coding avoids this upfront cost, making it ideal for short messages.
- **Convergence ($n \rightarrow \infty$):** As n increases, the two curves converge. The static overhead is amortized over more symbols, while the adaptive estimator converges to the true frequencies. The crossover point marks the length where the static header cost becomes negligible compared to the accumulated learning redundancy.

Ultimately, this demonstrates that adaptive coding achieves the same asymptotic efficiency as static coding without requiring a two-pass process.

4.4 Adaptive Coding with Memory

The experiment above used a 0-order model (assuming i.i.d.). To capture memory (where $H_\infty < H_1$), we would need to estimate conditional probabilities $P(X_t|X_{t-1})$. However, extending Eq. ?? to high-order contexts introduces **Context Dilution**. As context length increases, the total count n is split among $|\mathcal{X}|^k$ distinct contexts. Most contexts appear only a few times, so $n_{\text{context}} \approx 0$. In this regime, the bias δ dominates the counts ($n_x \ll \delta$), resulting in a near-uniform distribution and poor compression. Practical schemes like PPM solve this by “escaping” to shorter contexts when statistics are sparse.

5 Conclusion

This report demonstrated that while English text is a stationary source with memory, exploiting this theoretically (via H_∞) is hindered by sparse data. Adaptive coding offers a robust solution, allowing compression to approach the source entropy without prior knowledge of statistics. By deriving the Laplace estimator from Bayesian principles, we showed that adding a “pseudo-count” of 1 is not heuristic but the optimal strategy under a uniform prior assumption. This principle underpins modern compressors like LZ77 and prediction models in Large Language Models.

References

- [1] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006.
- [2] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.
- [3] S. Godsill, *Estimation Theory and Inference Methods*, Lecture Notes (3F3), University of Cambridge, 2023
- [4] J. Weisberg, “Laplace’s Rule of Succession,” *Formal Epistemology*, 2019. Available: <https://jonathanweisberg.org/post/inductive-logic-2/>
- [5] A. Tsun, “The Beta and Dirichlet Distributions,” Chapter 7.4 in *Probability & Statistics with Applications to Computing*, CS109 Course Reader, Stanford University, 2025. Available: https://web.stanford.edu/class/archive/cs/cs109/cs109.1218/files/student_drive/7.4.pdf
- [6] E. P. Xing, “Bayesian Nonparametrics: Dirichlet Processes,” Lecture 19 in *10-708: Probabilistic Graphical Models*, Lecture notes by C. Malings and J. Gao, Carnegie Mellon University, Spring 2014. Available: https://www.cs.cmu.edu/~epxing/Class/10708-14/scribe_notes/scribe_note_lecture19.pdf